Manual técnico y operación del sistema

Proyecto: SaborExpress | **Fecha de corte:** 14 de octubre de 2026 | **Versión:** 2.0 | **Tipo:** Documento técnico y académico

Estado de la evidencia: inspección documental y estática del código. Las pruebas de ejecución y los datos institucionales no aportados no se presumen realizados.

1. Visión operativa

La solución consta de repositorios independientes para API, web, móvil y documentación. Las fronteras entre interfaz, lógica y persistencia siguen una orientación feature-first: controladores delegan a servicios y repositorios; EF Core mantiene las entidades; Next y Expo presentan flujos por rol.

2. Referencia de carpetas y módulos

Repositorio

Ruta

Propósito	
Back	
Program.cs	
Bootstrap HTTP, auth, SignalR, servicios hospedados	
Back	
Configuration/	
Dependencias, CORS, JWT, DB y Swagger	
Back	
Controllers/	
Endpoints	
Back	
Modules/*/Services	
Reglas de aplicación	
Back	
Modules/*/Repositories	
Acceso de dominio a BD	
Back	
Data/Configurations	
EF Fluent API	
Back	
Migrations/	
Evolución relacional	
Web	
src/app	
Rutas Next.js	
Web	
src/features	
Servicios/hooks por feature	
Movil	
src/features	
Vistas/servicios/hooks	

Movil

app.json y eas.json

Configuración Expo y builds

3. Modelo de autenticación y autorización

El backend registra JWT Bearer, políticas de autorización y usuarios/roles. El rol debe comprobarse en servidor y complementarse con acceso a recurso concreto. ActiveShiftHandler introduce comprobaciones de turno en acciones designadas. Para login se identifican casos de bloqueo por intentos fallidos, correo sin confirmar y activación inicial. Asegurar expiración/revocación de sesiones.

4. Integraciones de datos

AppDbContext define DbSets por módulo y ejecuta configuraciones EF desde el ensamblado. DatabaseConfiguration.cs elige proveedor por valor literal; SQL Server, MySQL y PostgreSQL tienen diferencias semánticas, por lo que las migraciones requieren validación propia en el proveedor activo. Seeders iniciales: permisos, rol cliente y gerente inicial. Antes de usarlos en producción, revisar cuentas por defecto, reejecución y seguridad de secretos.

5. Integración de cobros

PaymentsController distingue pago manual, inicio Wompi para cliente, inicio QR de cajero, consulta por orden, sincronización y webhook externo. WompiPendingPaymentsSyncService aparece registrado en Program.cs. Operacionalmente, las referencias y transacciones deben reconciliarse sin duplicaciones. El body del webhook aparece registrado por el controlador; antes de operación real, evaluar si el log podría contener información sensible y aplicar filtrado apropiado.

6. Notificaciones y tiempo real

El Hub /hubs/notifications es parte de la API, no un microservicio aislado. Clientes web y móvil declaran @microsoft/signalr; se requiere probar negociación, reconexión, permisos de grupos/canales y coherencia con consulta REST. No se afirma una garantía de mensajes exactamente una vez.

7. Variables sensibles y despliegue

No guardar valores de ConnectionStrings, claves JWT, tokens Wompi, conexiones SMTP ni credenciales de servicio en el repositorio. Recomendación: appsettings.json sin secretos, .env.example redactado, pipeline con secret store, HTTPS, rotación de claves y administración de dominios permitidos.

8. Logs, monitoreo y soporte

Registrar ID de correlación, ruta, estado HTTP, duración y evento de pago sin datos financieros; separar logs de auditoría. Métricas a implementar: error rate, latencia p95, fallas de transacción, colas de pago pendiente, SignalR reconnections, fallos de migración, recuperación tras backup. Registrar incidentes con versión/commit.

9. Seguridad y mantenimiento

Área	Control recomendado	Evidencia a entregar
JWT/roles	Tests 401/403, acceso por objeto	Matriz negativa
Wompi	Validación firma, replay, duplicados	Sandbox con IDs censurados
Base de datos	Backup/restore y migración idempotente	Log de restauración
DevOps	Escaneo secrets/deps y CI	Reporte de workflow
Frontend	A11y, validación y errores	Checklist por pantalla
Puesta en producción	HTTPS, CORS restringido	

Config redactada revisada

10. Procedimiento de liberación sugerido

Crear rama → revisión por pares → compilar API/web/móvil → ejecutar pruebas unitarias/integración → verificar migrations dry run y sandbox de pagos → respaldar → desplegar en ventana acordada → smoke tests → monitorear → cerrar acta. Rollback debe restaurar binarios y estado coherente, no solo ejecutar git revert.

Fuentes primarias de implementación

API: https://github.com/yersonct/SaborExpressBack/tree/main

Web: https://github.com/yersonct/SaborExpressWeb/tree/main

Móvil: https://github.com/yersonct/SaborExpressMovil/tree/main

Referencia de presentación: https://github.com/jsgh1/SRD-v1/tree/dev/docs/docs-SRD

Control documental: documento elaborado a partir del estado observado en octubre de 2026. Antes de entregar, verificar el commit exacto, responsable, revisión del instructor y cualquier cambio posterior del código.